第 14 章

构建自定义的同步工具

类库包含了许多存在状态依赖性的类，例如 FutureTask、Semaphore 和 BlockingQueue 等。在这些类的一些操作中有着基于状态的前提条件，例如，不能从一个空的队列中删除元素，或者获取一个尚未结束的任务的计算结果，在这些操作可以执行之前，必须等到队列进入“非空”状态，或者任务进入“已完成”状态。

创建状态依赖类的最简单方法通常是在类库中现有状态依赖类的基础上进行构造。例如，在第 8 章的 ValueLatch 中就采用了这种方法，其中使用了一个 CountDownLatch 来提供所需的阻塞行为。但如果类库没有提供你需要的功能，那么还可以使用 Java 语言和类库提供的底层机制来构造自己的同步机制，包括内置的条件队列、显式的 Condition 对象以及 AbstractQueuedSynchronizer 框架。本章将介绍实现状态依赖性的各种选择，以及在使用平台提供的状态依赖性机制时需要遵守的各项规则。

14.1 状态依赖性的管理

在单线程程序中调用一个方法时，如果某个基于状态的前提条件未得到满足（例如“连接池必须非空”），那么这个条件将永远无法成真。因此，在编写顺序程序中的类时，要使得这些类在它们的前提条件未被满足时就失败。但在并发程序中，基于状态的条件可能会由于其他线程的操作而改变：一个资源池可能在几条指令之前还是空的，但现在却变为非空的，因为另一个线程可能会返回一个元素到资源池。对于并发对象上依赖状态的方法，虽然有时候在前提条件不满足的情况下不会失败，但通常有一种更好的选择，即等待前提条件变为真。

依赖状态的操作可以一直阻塞直到可以继续执行，这比使它们先失败再实现起来要更为方便且更不易出错。内置的条件队列可以使线程一直阻塞，直到对象进入某个进程可以继续执行的状态，并且当被阻塞的线程可以执行时再唤醒它们。我们将在 14.2 节介绍条件队列的详细内容，但为了突出高效的条件等待机制的价值，我们将首先介绍如何通过轮询与休眠等方式来（勉强地）解决状态依赖性问题。

可阻塞的状态依赖操作的形式如程序清单 14-1 所示。这种加锁模式有些不同寻常，因为锁是在操作的执行过程中被释放与重新获取的。构成前提条件的状态变量必须由对象的锁来保护，从而使它们在测试前提条件的同时保持不变。如果前提条件尚未满足，就必须释放锁，以便其他线程可以修改对象的状态，否则，前提条件就永远无法变成真。在再次测试前提条件之前，必须重新获得锁。

程序清单 14-1 可阻塞的状态依赖操作的结构

```
acquire lock on object state
while (precondition does not hold) {
    release lock
    wait until precondition might hold
    optionally fail if interrupted or timeout expires
    reacquire lock
}
perform action
    release lock
```

在生产者－消费者的设计中经常会使用像 ArrayBlockingQueue 这样的有界缓存。在有界缓存提供的 put 和 take 操作中都包含有一个前提条件：不能从空缓存中获取元素，也不能将元素放入已满的缓存中。当前提条件未满足时，依赖状态的操作可以抛出一个异常或返回一个错误状态（使其成为调用者的一个问题），也可以保持阻塞直到对象进入正确的状态。

接下来介绍有界缓存的几种实现，其中将采用不同的方法来处理前提条件失败的问题。在每种实现中都扩展了程序清单 14-2 中的 BaseBoundedBuffer，在这个类中实现了一个基于数组的循环缓存，其中各个缓存状态变量（buf、head、tail 和 count）均由缓存的内置锁来保护。它还提供了同步的 doPut 和 doTake 方法，并在子类中通过这些方法来实现 put 和 take 操作，底层的状态将对子类隐藏。

程序清单 14-2 有界缓存实现的基类

```
@ThreadSafe
public abstract class BaseBoundedBuffer<V> {
    @GuardedBy("this") private final V[] buf;
    @GuardedBy("this") private int tail;
    @GuardedBy("this") private int head;
    @GuardedBy("this") private int count;

    protected BaseBoundedBuffer(int capacity) {
        this.buf = (V[]) new Object[capacity];
    }

    protected synchronized final void doPut(V v) {
        buf[tail] = v;
        if (++tail == buf.length)
            tail = 0;
        ++count;
    }

    protected synchronized final V doTake() {
        V v = buf[head];
        buf[head] = null;
        if (++head == buf.length)
            head = 0;
        --count;
        return v;
    }
```

```
    public synchronized final boolean isFull() {
        return count == buf.length;
    }

    public synchronized final boolean isEmpty() {
        return count == 0;
    }
}
```

14.1.1 示例：将前提条件的失败传递给调用者

程序清单 14-3 的 GrumpyBoundedBuffer 是第一个简单的有界缓存实现。put 和 take 方法都进行了同步以确保实现对缓存状态的独占访问，因为这两个方法在访问缓存时都采用“先检查再运行”的逻辑策略。

程序清单 14-3 当不满足前提条件时，有界缓存不会执行相应的操作

```
@ThreadSafe
public class GrumpyBoundedBuffer<V> extends BaseBoundedBuffer<V> {
    public GrumpyBoundedBuffer(int size) { super(size); }

    public synchronized  void put(V v) throws BufferFullException {
        if (isFull())
            throw new BufferFullException();
        doPut(v);
    }

    public synchronized  V take() throws BufferEmptyException {
        if (isEmpty())
            throw new BufferEmptyException();
        return doTake();
    }
}
```

尽管这种方法实现起来很简单，但使用起来却并非如此。异常应该用于发生异常条件的情况中 [EJ Item 39]。“缓存已满”并不是有界缓存的一个异常条件，就像“红灯”并不表示交通信号灯出现了异常。在实现缓存时得到的简化（使调用者管理状态依赖性）并不能抵消在使用时存在的复杂性，因为现在调用者必须做好捕获异常的准备，并且在每次缓存操作时都需要重试[⊖]。程序清单 14-4 给出了对 take 的调用——并不是很漂亮，尤其是当程序中有许多地方都调用 put 和 take 方法时。

程序清单 14-4 调用 GrumpyBoundedBuffer 的代码

```
while (true) {
    try {
        V item = buffer.take();
        // 对于 item 执行一些操作
```

⊖ 如果将状态依赖性交给调用者管理，那么将导致一些功能无法实现，例如维持 FIFO 顺序，由于迫使调用者重试，因此失去了“谁先到达”的信息。

```
            break;
        } catch (BufferEmptyException e) {
            Thread.sleep(SLEEP_GRANULARITY);
        }
    }
```

这种方法的一种变化形式是，当缓存处于某种错误的状态时返回一个错误值。这是一种改进，因为并没有放弃异常机制，抛出的异常意味着“对不起，请再试一次”，但这种方法并没有解决根本问题：调用者必须自行处理前提条件失败的情况[⊖]。

程序清单 14-4 中的客户代码不是实现重试的唯一方式。调用者可以不进入休眠状态，而直接重新调用 take 方法，这种方法被称为忙等待或自旋等待。如果缓存的状态在很长一段时间内都不会发生变化，那么使用这种方法就会消耗大量的 CPU 时间。但是，调用者也可以进入休眠状态来避免消耗过多的 CPU 时间，但如果缓存的状态在刚调用完 sleep 就立即发生变化，那么将不必要地休眠一段时间。因此，客户代码必须要在二者之间进行选择：要么容忍自旋导致的 CPU 时钟周期浪费，要么容忍由于休眠而导致的低响应性。（除了忙等待与休眠之外，还有一种选择就是调用 Thread.yield，这相当于给调度器一个提示：现在需要让出一定的时间使另一个线程运行。假如正在等待另一个线程执行工作，那么如果选择让出处理器而不是消耗完整个 CPU 调度时间片，那么可以使整体的执行过程变快。）

14.1.2 示例：通过轮询与休眠来实现简单的阻塞

程序清单 14-5 中的 SleepyBoundedBuffer 尝试通过 put 和 take 方法来实现一种简单的“轮询与休眠”重试机制，从而使调用者无须在每次调用时都实现重试逻辑。如果缓存为空，那么 take 将休眠并直到另一个线程在缓存中放入一些数据；如果缓存是满的，那么 put 将休眠并直到另一个线程从缓存中移除一些数据，以便有空间容纳新的数据。这种方法将前提条件的管理操作封装起来，并简化了对缓存的使用——这正是朝着正确的改进方向迈出了一步。

程序清单 14-5 使用简单阻塞实现的有界缓存

```
@ThreadSafe
public class SleepyBoundedBuffer<V> extends BaseBoundedBuffer<V> {
    public SleepyBoundedBuffer(int size) { super(size); }

    public void put(V v) throws InterruptedException {
        while (true) {
            synchronized (this) {
                if (!isFull()) {
                    doPut(v);
                    return;
                }
            }
```

⊖ Queue 提供了上述两种选择，即 poll 方法能够在队列为空时返回 null，而 remove 方法则抛出一个异常，但 Queue 并不适合在生产者 - 消费者设计中使用。BlockingQueue 中的操作只有当队列处于正确状态时才会进行处理，否则将阻塞，因此当生产者和消费者并发执行时，BlockingQueue 才是更好的选择。

```
            Thread.sleep(SLEEP_GRANULARITY);
        }
    }

    public V take() throws InterruptedException {
        while (true) {
            synchronized (this) {
                if (!isEmpty())
                    return doTake();
            }
            Thread.sleep(SLEEP_GRANULARITY);
        }
    }
}
```

SleepyBoundedBuffer 的实现远比之前的实现复杂[⊖]。缓存代码必须在持有缓存锁的时候才能测试相应的状态条件，因为表示状态条件的变量是由缓存锁保护的。如果测试失败，那么当前执行的线程将首先释放锁并休眠一段时间，从而使其他线程能够访问缓存[⊖]。当线程醒来时，它将重新请求锁并再次尝试执行操作，因而线程将反复地在休眠以及测试状态条件等过程之间进行切换，直到可以执行操作为止。

从调用者的角度看，这种方法能很好地运行，如果某个操作可以执行，那么就立即执行，否则就阻塞，调用者无须处理失败和重试。要选择合适的休眠时间间隔，就需要在响应性与 CPU 使用率之间进行权衡。休眠的间隔越小，响应性就越高，但消耗的 CPU 资源也越高。图 14-1 给出了休眠间隔对响应性的影响：在缓存中出现可用空间的时刻与线程醒来并再次检查的时刻之间可能存在延迟。

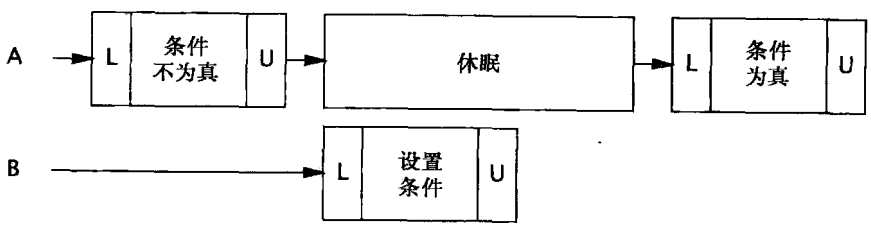

图 14-1　在线程刚刚进入休眠后，条件立即变为真，此时将存在不必要的休眠时间

SleepyBoundedBuffer 对调用者提出了一个新的需求：处理 InterruptedException。当一个方法由于等待某个条件变成真而阻塞时，需要提供一种取消机制（请参见第 7 章）。与大多数具备良好行为的阻塞库方法一样，SleepyBoundedBuffer 通过中断来支持取消，如果该方法被中断，那么将提前返回并抛出 InterruptedException。

⊖ 与白雪公主中其他 5 个小矮人的名字相对应的其他有界缓存留给读者来实现，尤其是 SneezyBoundedBuffer。（白雪公主中 7 个小矮人的名字分别为 Sleepy，Grumpy，Sneezy，Doc，Bashful，Happy 和 Dopey，其中与 Sleepy 和 Grumpy 对应的 BoundedBuffer 已经实现了。）

⊖ 通常，如果线程在休眠或者被阻塞时持有一个锁，那么这通常是一种不好的做法，因为只要线程不释放这个锁，有些条件（缓存为满 / 空）就永远无法为真。

这种通过轮询与休眠来实现阻塞操作的过程需要付出大量的努力。如果存在某种挂起线程的方法，并且这种方法能够确保当某个条件成真时线程立即醒来，那么将极大地简化实现工作。这正是条件队列实现的功能。

14.1.3 条件队列

条件队列就好像烤面包机中通知“面包已烤好”的铃声。如果你注意听着铃声，那么当面包烤好后可以立刻得到通知，然后放下手头的事情（或者先把手头的事情做完，例如先看完报纸）开始品尝面包。如果没有听见铃声（可能出去拿报纸了），那么会错过通知信息，但回到厨房时还可以观察烤面包机的状态，如果已经烤好，那么就取出面包，如果还未烤好，就再次留意铃声。

“条件队列”这个名字来源于：它使得一组线程（称之为等待线程集合）能够通过某种方式来等待特定的条件变成真。传统队列的元素是一个个数据，而与之不同的是，条件队列中的元素是一个个正在等待相关条件的线程。

正如每个 Java 对象都可以作为一个锁，每个对象同样可以作为一个条件队列，并且 Object 中的 wait、notify 和 notifyAll 方法就构成了内部条件队列的 API。对象的内置锁与其内部条件队列是相互关联的，要调用对象 X 中条件队列的任何一个方法，必须持有对象 X 上的锁。这是因为“等待由状态构成的条件”与“维护状态一致性”这两种机制必须被紧密地绑定在一起：只有能对状态进行检查时，才能在某个条件上等待，并且只有能修改状态时，才能从条件等待中释放另一个线程。

Object.wait 会自动释放锁，并请求操作系统挂起当前线程，从而使其他线程能够获得这个锁并修改对象的状态。当被挂起的线程醒来时，它将在返回之前重新获取锁。从直观上来理解，调用 wait 意味着“我要去休息了，但当发生特定的事情时唤醒我”，而调用通知方法就意味着“特定的事情发生了”。

在程序清单 14-6 的 BoundedBuffer 中使用了 wait 和 notifyAll 来实现一个有界缓存。这比使用“休眠”的有界缓存更简单，并且更高效（当缓存状态没有发生变化时，线程醒来的次数将更少），响应性也更高（当发生特定状态变化时将立即醒来）。这是一个较大的改进，但要注意：与使用“休眠”的有界缓存相比，条件队列并没有改变原来的语义。它只是在多个方面进行了优化：CPU 效率、上下文切换开销和响应性等。如果某个功能无法通过“轮询和休眠”来实现，那么使用条件队列也无法实现[⊖]，但条件队列使得在表达和管理状态依赖性时更加简单和高效。

程序清单 14-6 使用条件队列实现的有界缓存

```
@ThreadSafe
public class BoundedBuffer<V> extends BaseBoundedBuffer<V> {
    // 条件谓词：not-full (!isFull())
```

⊖ 这并非完全正确；一个公平的条件队列可以确保线程按照顺序从等待集合中释放。与内置锁相同，内置条件队列并不提供公平的排队操作，而在显式的 Condition 却可以提供公平或非公平的排队操作。

```
    // 条件谓词：not-empty (!isEmpty())

    public BoundedBuffer(int size) { super(size); }

    // 阻塞并直到：not-full
    public synchronized void put(V v) throws InterruptedException {
        while (isFull())
            wait();
        doPut(v);
        notifyAll();
    }

    // 阻塞并直到：not-empty
    public synchronized V take() throws InterruptedException {
        while (isEmpty())
            wait();
        V v = doTake();
        notifyAll();
        return v;
    }
}
```

最终，BoundedBuffer 变得足够好了，不仅简单易用，而且实现了明晰的状态依赖性管理[⊖]。在产品的正式版本中还应包括限时版本的 put 和 take，这样当阻塞操作不能在预计时间内完成时，可以因超时而返回。通过使用定时版本的 Object.wait，可以很容易实现这些方法。

14.2 使用条件队列

条件队列使构建高效以及高可响应性的状态依赖类变得更容易，但同时也很容易被不正确地使用。虽然许多规则都能确保正确地使用条件队列，但在编译器或系统平台上却并没有强制要求遵循这些规则。（这也是为什么要尽量基于 LinkedBlockingQueue、Latch、Semaphore 和 FutureTask 等类来构造程序的原因之一，如果能避免使用条件队列，那么实现起来将容易许多。）

14.2.1 条件谓词

要想正确地使用条件队列，关键是找出对象在哪个条件谓词上等待。条件谓词将在等待与通知等过程中导致许多困惑，因为在 API 中没有对条件谓词进行实例化的方法，并且在 Java 语言规范或 JVM 实现中也没有任何信息可以确保正确地使用它们。事实上，在 Java 语言规范或 Javadoc 中根本就没有直接提到它。但如果没有条件谓词，条件等待机制将无法发挥作用。

条件谓词是使某个操作成为状态依赖操作的前提条件。在有界缓存中，只有当缓存不为空时，take 方法才能执行，否则必须等待。对 take 方法来说，它的条件谓词就是“缓存不为空”，take 方法在执行之前必须首先测试该条件谓词。同样，put 方法的条件谓词是“缓存不满”。条件谓词是由类中各个状态变量构成的表达式。BaseBoundedBuffer 在测试“缓存不为空”时将

⊖ 14.3 节的 ConditionBoundedBuffer 还要更好，因为它能使用单一通知方法而不是 notifyAll，因此效率更高。

把 count 与 0 进行比较，在测试“缓存不满”时将把 count 与缓存的大小进行比较。

> 将与条件队列相关联的条件谓词以及在这些条件谓词上等待的操作都写入文档。

在条件等待中存在一种重要的三元关系，包括加锁、wait 方法和一个条件谓词。在条件谓词中包含多个状态变量，而状态变量由一个锁来保护，因此在测试条件谓词之前必须先持有这个锁。锁对象与条件队列对象（即调用 wait 和 notify 等方法所在的对象）必须是同一个对象。

在 BoundedBuffer 中，缓存的状态是由缓存锁保护的，并且缓存对象被用做条件队列。take 方法将获取请求缓存锁，然后对条件谓词（即缓存为非空）进行测试。如果缓存非空，那么它会移除第一个元素。之所以能这样做，是因为 take 此时仍然持有保护缓存状态的锁。

如果条件谓词不为真（缓存为空），那么 take 必须等待并直到另一个线程在缓存中放入一个对象。take 将在缓存的内置条件队列上调用 wait 方法，这需要持有条件队列对象上的锁。这是一种谨慎的设计，因为 take 方法已经持有在测试条件谓词时（并且如果条件谓词为真，那么在同一个原子操作中修改缓存的状态）需要的锁。wait 方法将释放锁，阻塞当前线程，并等待直到超时，然后线程被中断或者通过一个通知被唤醒。在唤醒进程后，wait 在返回前还要重新获取锁。当线程从 wait 方法中被唤醒时，它在重新请求锁时不具有任何特殊的优先级，而要与任何其他尝试进入同步代码块的线程一起正常地在锁上进行竞争。

> 每一次 wait 调用都会隐式地与特定的条件谓词关联起来。当调用某个特定条件谓词的 wait 时，调用者必须已经持有与条件队列相关的锁，并且这个锁必须保护着构成条件谓词的状态变量。

14.2.2 过早唤醒

虽然在锁、条件谓词和条件队列之间的三元关系并不复杂，但 wait 方法的返回并不一定意味着线程正在等待的条件谓词已经变成真了。

内置条件队列可以与多个条件谓词一起使用。当一个线程由于调用 notifyAll 而醒来时，并不意味该线程正在等待的条件谓词已经变成真了。（这就像烤面包机和咖啡机共用一个铃声，当响铃后，你必须查看是哪个设备发出的铃声。）[⊖]另外，wait 方法还可以“假装”返回，而不是由于某个线程调用了 notify。[⊜]

当执行控制重新进入调用 wait 的代码时，它已经重新获取了与条件队列相关联的锁。现在条件谓词是不是已经变为真了？或许。在发出通知的线程调用 notifyAll 时，条件谓词可能已经变成真，但在重新获取锁时将再次变为假。在线程被唤醒到 wait 重新获取锁的这段时间里，可能有其他线程已经获取了这个锁，并修改了对象的状态。或者，条件谓词从调用 wait 起根本就

⊖ 在 Tim（Tim Peierls，本书的合著者之一）的厨房里就是这种情况，当听到一个铃声时，可能有很多设备正在发出声音，此时必须检查烤面包机、微波炉、咖啡机和其他设备来判断是哪一个设备发出的铃声。

⊜ 继续以“早餐”为例，这就好比烤面包机的线路连接有问题，有时候当面包还未烤好时，铃声就响起来了。

没有变成真。你并不知道另一个线程为什么调用 notify 或 notifyAll，也许是因为与同一条件队列相关的另一个条件谓词变成了真。“一个条件队列与多个条件谓词相关”是一种很常见的情况——在 BoundedBuffer 中使用的条件队列与“非满”和“非空”两个条件谓词相关。[⊖]

基于所有这些原因，每当线程从 wait 中唤醒时，都必须再次测试条件谓词，如果条件谓词不为真，那么就继续等待（或者失败）。由于线程在条件谓词不为真的情况下也可以反复地醒来，因此必须在一个循环中调用 wait，并在每次迭代中都测试条件谓词。程序清单 14-7 给出了条件等待的标准形式。

程序清单 14-7　状态依赖方法的标准形式

```
void stateDependentMethod() throws InterruptedException {
    // 必须通过一个锁来保护条件谓词
    synchronized(lock) {
        while (!conditionPredicate())
            lock.wait();
        // 现在对象处于合适的状态
    }
}
```

当使用条件等待时（例如 Object.wait 或 Condition.await）：

- 通常都有一个条件谓词——包括一些对象状态的测试，线程在执行前必须首先通过这些测试。
- 在调用 wait 之前测试条件谓词，并且从 wait 中返回时再次进行测试。
- 在一个循环中调用 wait。
- 确保使用与条件队列相关的锁来保护构成条件谓词的各个状态变量。
- 当调用 wait、notify 或 notifyAll 等方法时，一定要持有与条件队列相关的锁。
- 在检查条件谓词之后以及开始执行相应的操作之前，不要释放锁。

14.2.3　丢失的信号

第 10 章曾经讨论过活跃性故障，例如死锁和活锁。另一种形式的活跃性故障是丢失的信号。丢失的信号是指：线程必须等待一个已经为真的条件，但在开始等待之前没有检查条件谓词。现在，线程将等待一个已经发过的事件。这就好比在启动了烤面包机后出去拿报纸，当你还在屋外时烤面包机的铃声响了，但你没有听到，因此还会坐在厨房的桌子前等着烤面包机的铃声。你可能会等待很长的时间[⊜]。通知并不像你涂在面包上的果酱，它没有“黏附性”。如果线程 A 通知了一个条件队列，而线程 B 随后在这个条件队列上等待，那么线程 B 将不会立即

⊖ 线程可能同时在“非满”与“非空”这两个条件谓词上等待。当生产者 / 消费者的数量超过缓存的容量时，就会出现这种情况。

⊜ 为了摆脱等待，其他人也不得不开始烤面包，从而使情况变得更糟，当铃声响起时，还要与别人争论这个面包是属于谁的。

醒来，而是需要另一个通知来唤醒它。像上述程序清单中警示之类的编码错误（例如，没有在调用 wait 之前检测条件谓词）就会导致信号的丢失。如果按照程序清单 14-7 的方式来设计条件等待，那么就不会发生信号丢失的问题。

14.2.4 通知

到目前为止，我们介绍了条件等待的前一半内容：等待。另一半内容是通知。在有界缓存中，如果缓存为空，那么在调用 take 时将阻塞。在缓存变为非空时，为了使 take 解除阻塞，必须确保在每条使缓存变为非空的代码路径中都发出一个通知。在 BoundedBuffer 中，只有一条代码路径，即在 put 方法之后。因此，put 在成功地将一个元素添加到缓存后，将调用 notifyAll。同样，take 在移除一个元素后也将调用 notifyAll，向任何正在等待“不为满”条件的线程发出通知：缓存已经不满了。

每当在等待一个条件时，一定要确保在条件谓词变为真时通过某种方式发出通知。

在条件队列 API 中有两个发出通知的方法，即 notify 和 notifyAll。无论调用哪一个，都必须持有与条件队列对象相关联的锁。在调用 notify 时，JVM 会从这个条件队列上等待的多个线程中选择一个来唤醒，而调用 notifyAll 则会唤醒所有在这个条件队列上等待的线程。由于在调用 notify 或 notifyAll 时必须持有条件队列对象的锁，而如果这些等待中线程此时不能重新获得锁，那么无法从 wait 返回，因此发出通知的线程应该尽快地释放锁，从而确保正在等待的线程尽可能快地解除阻塞。

由于多个线程可以基于不同的条件谓词在同一个条件队列上等待，因此如果使用 notify 而不是 notifyAll，那么将是一种危险的操作，因为单一的通知很容易导致类似于信号丢失的问题。

在 BoundedBuffer 中很好地说明为什么在大多数情况下应该优先选择 notifyAll 而不是单个的 notify。这里的条件队列用于两个不同的条件谓词：“非空”和“非满”。假设线程 A 在条件队列上等待条件谓词 PA，同时线程 B 在同一个条件队列上等待条件谓词 PB。现在，假设 PB 变成真，并且线程 C 执行一个 notify：JVM 将从它拥有的众多线程中选择一个并唤醒。如果选择了线程 A，那么它被唤醒，并且看到 PA 尚未变成真，因此将继续等待。同时，线程 B 本可以开始执行，却没有被唤醒。这并不是严格意义上的“丢失信号”，而更像一种“被劫持的”信号，但导致的问题是相同的：线程正在等待一个已经（或者本应该）发生过的信号。

只有同时满足以下两个条件时，才能用单一的 notify 而不是 notifyAll：

所有等待线程的类型都相同。只有一个条件谓词与条件队列相关，并且每个线程在从 wait 返回后将执行相同的操作。

单进单出。在条件变量上的每次通知，最多只能唤醒一个线程来执行。

BoundedBuffer 满足“单进单出”的条件，但不满足“所有等待线程的类型都相同”的条件，因此正在等待的线程可能在等待“非满”，也可能在等待“非空”。例如第 5 章的 TestHarness 中使用的“开始阀门”闭锁（单个事件释放一组线程）并不满足“单进单出”的需求，因为这个“开始阀门”将使得多个线程开始执行。

由于大多数类并不满足这些需求，因此普遍认可的做法是优先使用 notifyAll 而不是 notify。虽然 notifyAll 可能比 notify 更低效，但却更容易确保类的行为是正确的。

有些开发人员并不赞同这种“普遍认可的做法”。当只有一个线程可以执行时，如果使用 notifyAll，那么将是低效的，这种低效情况带来的影响有时候很小，但有时侯却非常大。如果有 10 个线程在一个条件队列上等待，那么调用 notifyAll 将唤醒每个线程，并使得它们在锁上发生竞争。然后，它们中的大多数或者全部又都回到休眠状态。因而，在每个线程执行一个事件的同时，将出现大量的上下文切换操作以及发生竞争的锁获取操作。（最坏的情况是，在使用 notifyAll 时将导致 $O(n^2)$ 次唤醒操作，而实际上只需要 n 次唤醒操作就足够了）。这是“性能考虑因素与安全性考虑因素相互矛盾”的另一种情况。

在 BoundedBuffer 的 put 和 take 方法中采用的通知机制是保守的：每当将一个对象放入缓存或者从缓存中移走一个对象时，就执行一次通知。我们可以对其进行优化：首先，仅当缓存从空变为非空，或者从满转为非满时，才需要释放一个线程。并且，仅当 put 或 take 影响到这些状态转换时，才发出通知。这也被称为“条件通知（Conditional Notification）。虽然“条件通知”可以提升性能，但却很难正确地实现（而且还会使子类的实现变得复杂），因此在使用时应该谨慎。程序清单 14-8 给出了如何在 BoundedBuffer.put 中使用“条件通知”。

程序清单 14-8 在 BoundedBuffer.put 中使用条件通知

```
public synchronized void put(V v) throws InterruptedException {
    while (isFull())
        wait();
    boolean wasEmpty = isEmpty();
    doPut(v);
    if (wasEmpty)
        notifyAll();
}
```

单次通知和条件通知都属于优化措施。通常，在使用这些优化措施时，应该遵循“首选使程序正确地执行，然后才使其运行得更快”这个原则。如果不正确地使用这些优化措施，那么很容易在程序中引入奇怪的活跃性故障。

14.2.5 示例：阀门类

在第 5 章的 TestHarness 中使用的“开始阀门闭锁”在初始化时指定的参数为 1，从而创建了一个二元闭锁：它只有两种状态，即初始状态和结束状态。闭锁能阻止线程通过开始阀门，并直到阀门被打开，此时所有的线程都可以通过该阀门。虽然闭锁机制通常都能满足需求，但在某些情况下存在一个缺陷：按照这种方式构造的阀门在打开后无法重新关闭。

通过使用条件等待，可以很容易地开发一个可重新关闭的 ThreadGate 类，如程序清

单 14-9 所示。ThreadGate 可以打开和关闭阀门，并提供一个 await 方法，该方法能一直阻塞直到阀门被打开。在 open 方法中使用了 notifyAll，这是因为这个类的语义不满足单次通知的“单进单出”测试。

程序清单 14-9 使用 wait 和 notifyAll 来实现可重新关闭的阀门

```
@ThreadSafe
public class ThreadGate {
    // 条件谓词：opened-since(n) (isOpen || generation>n)
    @GuardedBy("this") private boolean isOpen;
    @GuardedBy("this") private int generation;

    public synchronized void close() {
        isOpen = false;
    }

    public synchronized void open() {
        ++generation;
        isOpen = true;
        notifyAll();
    }

    // 阻塞并直到：opened-since(generation on entry)
    public synchronized void await() throws InterruptedException {
        int arrivalGeneration = generation;
        while (!isOpen && arrivalGeneration == generation)
            wait();
    }
}
```

在 await 中使用的条件谓词比测试 isOpen 复杂得多。这种条件谓词是必需的，因为如果当阀门打开时有 N 个线程正在等待它，那么这些线程都应该被允许执行。然而，如果阀门在打开后又非常快速地关闭了，并且 await 方法只检查 isOpen，那么所有线程都可能无法释放：当所有线程收到通知时，将重新请求锁并退出 wait，而此时的阀门可能已经再次关闭了。因此，在 ThreadGate 中使用了一个更复杂的条件谓词：每次阀门关闭时，递增一个“Generation”计数器，如果阀门现在是打开的，或者阀门自从该线程到达后就一直是打开的，那么线程就可以通过 await。

由于 ThreadGate 只支持等待打开阀门，因此它只在 open 中执行通知。要想既支持“等待打开”又支持“等待关闭”，那么 ThreadGate 必须在 open 和 close 中都进行通知。这很好地说明了为什么在维护状态依赖的类时是非常困难的——当增加一个新的状态依赖操作时，可能需要对多条修改对象的代码路径进行改动，才能正确地执行通知。

14.2.6 子类的安全问题

在使用条件通知或单次通知时，一些约束条件使得子类化过程变得更加复杂 [CPJ 3.3.3.3]。要想支持子类化，那么在设计类时需要保证：如果在实施子类化时违背了条件通知或单次通知的某个需求，那么在子类中可以增加合适的通知机制来代表基类。

对于状态依赖的类，要么将其等待和通知等协议完全向子类公开（并且写入正式文档），要么完全阻止子类参与到等待和通知等过程中。（这是对“要么围绕着继承来设计和文档化，要么禁止使用继承”这条规则的一种扩展 [EJ Item 15]。）当设计一个可被继承的状态依赖类时，至少需要公开条件队列和锁，并且将条件谓词和同步策略都写入文档。此外，还可能需要公开一些底层的状态变量。（最糟糕的情况是，一个状态依赖的类虽然将其状态向子类公开，但却没有将相应的等待和通知等协议写入文档，这就类似于一个类虽然公开了它的状态变量，但却没有将其不变性条件写入文档。）

另外一种选择就是完全禁止子类化，例如将类声明为 final 类型，或者将条件队列、锁和状态变量等隐藏起来，使子类看不见它们。否则，如果子类破坏了在基类中使用 notify 的方式，那么基类需要修复这种破坏。考虑一个无界的可阻塞栈，当栈为空时，pop 操作将阻塞，但 push 操作通常可以执行。这就满足了使用单次通知的需求。如果在这个类中使用了单次通知，并且在其一个子类中添加了一个阻塞的“弹出两个连续元素”方法，那么就会出现两种类型的等待线程：等待弹出一个元素的线程和等待弹出两个元素的线程。但如果基类将条件队列公开出来，并且将使用该条件队列的协议也写入文档，那么子类就可以将 push 方法改写为执行 notifyAll，从而重新确保安全性。

14.2.7 封装条件队列

通常，我们应该把条件队列封装起来，因而除了使用条件队列的类，就不能在其他地方访问它。否则，调用者会自以为理解了在等待和通知上使用的协议，并且采用一种违背设计的方式来使用条件队列。（除非条件队列对象对于你无法控制的代码来说是不可访问的，否则就不可能要求在单次通知中的所有等待线程都是同一类型的。如果外部代码错误地在条件队列上等待，那么可能通知协议，并导致一个“被劫持的”信号。）

不幸的是，这条建议——将条件队列对象封装起来，与线程安全类的最常见设计模式并不一致，在这种模式中建议使用对象的内置锁来保护对象自身的状态。在 BoundedBuffer 中给出了这种常见的模式，即缓存对象自身既是锁，又是条件队列。然而，可以很容易将 BoundedBuffer 重新设计为使用私有的锁对象和条件队列，唯一的不同之处在于，新的 BoundedBuffer 不再支持任何形式的客户端加锁。

14.2.8 入口协议与出口协议

Wellings (Wellings，2004) 通过“入口协议和出口协议（Entry and Exit Protocols）”来描述 wait 和 notify 方法的正确使用。对于每个依赖状态的操作，以及每个修改其他操作依赖状态的操作，都应该定义一个入口协议和出口协议。入口协议就是该操作的条件谓词，出口协议则包括，检查被该操作修改的所有状态变量，并确认它们是否使某个其他的条件谓词变为真，如果是，则通知相关的条件队列。

在 AbstractQueuedSynchronizer（java.util.concurrent 包中大多数依赖状态的类都是基于这个类构建的）中使用出口协议（请参见 14.4 节）。这个类并不是由同步器类执行自己的通知，

而是要求同步器方法返回一个值来表示该类的操作是否已经解除了一个或多个等待线程的阻塞。这种明确的 API 调用需求使得更难以“忘记”在某些状态转换发生时进行通知。

14.3 显式的 Condition 对象

第 13 章曾介绍过，在某些情况下，当内置锁过于灵活时，可以使用显式锁。正如 Lock 是一种广义的内置锁，Condition（参见程序清单 14-10）也是一种广义的内置条件队列。

程序清单 14-10 Condition 接口

```
public interface Condition {
    void await() throws InterruptedException;
    boolean await(long time, TimeUnit unit)
            throws InterruptedException;
    long awaitNanos(long nanosTimeout) throws InterruptedException;
    void awaitUninterruptibly();
    boolean awaitUntil(Date deadline) throws InterruptedException;

    void signal();
    void signalAll();
}
```

内置条件队列存在一些缺陷。每个内置锁都只能有一个相关联的条件队列，因而在像 BoundedBuffer 这种类中，多个线程可能在同一个条件队列上等待不同的条件谓词，并且在最常见的加锁模式下公开条件队列对象。这些因素都使得无法满足在使用 notifyAll 时所有等待线程为同一类型的需求。如果想编写一个带有多个条件谓词的并发对象，或者想获得除了条件队列可见性之外的更多控制权，就可以使用显式的 Lock 和 Condition 而不是内置锁和条件队列，这是一种更灵活的选择。

一个 Condition 和一个 Lock 关联在一起，就像一个条件队列和一个内置锁相关联一样。要创建一个 Condition，可以在相关联的 Lock 上调用 Lock.newCondition 方法。正如 Lock 比内置加锁提供了更为丰富的功能，Condition 同样比内置条件队列提供了更丰富的功能：在每个锁上可存在多个等待、条件等待可以是可中断的或不可中断的、基于时限的等待，以及公平的或非公平的队列操作。

与内置条件队列不同的是，对于每个 Lock，可以有任意数量的 Condition 对象。Condition 对象继承了相关的 Lock 对象的公平性，对于公平的锁，线程会依照 FIFO 顺序从 Condition.await 中释放。

> **特别注意：在 Condition 对象中，与 wait、notify 和 notifyAll 方法对应的分别是 await、signal 和 signalAll。但是，Condition 对 Object 进行了扩展，因而它也包含 wait 和 notify 方法。一定要确保使用正确的版本——await 和 signal。**

程序清单 14-11 给出了有界缓存的另一种实现，即使用两个 Condition，分别为 notFull 和 notEmpty，用于表示“非满”与“非空”两个条件谓词。当缓存为空时，take 将阻塞并等待

notEmpty，此时 put 向 notEmpty 发送信号，可以解除任何在 take 中阻塞的线程。

程序清单 14-11 使用显式条件变量的有界缓存

```
@ThreadSafe
public class ConditionBoundedBuffer<T> {
    protected final Lock lock = new ReentrantLock();
    // 条件谓词：notFull (count < items.length)
    private final Condition notFull    = lock.newCondition();
    // 条件谓词：notEmpty (count > 0)
    private final Condition notEmpty  = lock.newCondition();
    @GuardedBy("lock")
    private final T[] items = (T[]) new Object[BUFFER_SIZE];
    @GuardedBy("lock") private int tail, head, count;

    // 阻塞并直到：notFull
    public void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length)
                notFull.await();
            items[tail] = x;
            if (++tail == items.length)
                tail = 0;
            ++count;
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    // 阻塞并直到：notEmpty
    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (count == 0)
                notEmpty.await();
            T x = items[head];
            items[head] = null;
            if (++head == items.length)
                head = 0;
            --count;
            notFull.signal();
            return x;
        } finally {
            lock.unlock();
        }
    }
}
```

ConditionBoundedBuffer 的行为和 BoundedBuffer 相同，但它对条件队列的使用方式更容易理解——在分析使用多个 Condition 的类时，比分析一个使用单一内部队列加多个条件谓词的类简单得多。通过将两个条件谓词分开并放到两个等待线程集中，Condition 使其更容易满足单次通知的需求。signal 比 signalAll 更高效，它能极大地减少在每次缓存操作中发生的上下文

切换与锁请求的次数。

与内置锁和条件队列一样，当使用显式的 Lock 和 Condition 时，也必须满足锁、条件谓词和条件变量之间的三元关系。在条件谓词中包含的变量必须由 Lock 来保护，并且在检查条件谓词以及调用 await 和 signal 时，必须持有 Lock 对象[⊖]。

在使用显式的 Condition 和内置条件队列之间进行选择时，与在 ReentrantLock 和 synchronized 之间进行选择是一样的：如果需要一些高级功能，例如使用公平的队列操作或者在每个锁上对应多个等待线程集，那么应该优先使用 Condition 而不是内置条件队列。（如果需要 ReentrantLock 的高级功能，并且已经使用了它，那么就已经做出了选择。）

14.4 Synchronizer 剖析

在 ReentrantLock 和 Semaphore 这两个接口之间存在许多共同点。这两个类都可以用做一个“阀门”，即每次只允许一定数量的线程通过，并当线程到达阀门时，可以通过（在调用 lock 或 acquire 时成功返回），也可以等待（在调用 lock 或 acquire 时阻塞），还可以取消（在调用 tryLock 或 tryAcquire 时返回“假”，表示在指定的时间内锁是不可用的或者无法获得许可）。而且，这两个接口都支持可中断的、不可中断的以及限时的获取操作，并且也都支持等待线程执行公平或非公平的队列操作。

列出了这种共性后，你或许会认为 Semaphore 是基于 ReentrantLock 实现的，或者认为 ReentrantLock 实际上是带有一个许可的 Semaphore。这些实现方式都是可行的，一个很常见的练习就是，证明可以通过锁来实现计数信号量（如程序清单 14-12 中的 SemaphoreOnLock 所示），以及可以通过计数信号量来实现锁。

程序清单 14-12 使用 Lock 来实现信号量

```
// 并非 java.util.concurrent.Semaphore 的真实实现方式
@ThreadSafe
public class SemaphoreOnLock {
    private final Lock lock = new ReentrantLock();
    // 条件谓词：permitsAvailable (permits > 0)
    private final Condition permitsAvailable = lock.newCondition();
    @GuardedBy("lock") private int permits;

    SemaphoreOnLock(int initialPermits) {
        lock.lock();
        try {
            permits = initialPermits;
        } finally {
            lock.unlock();
        }
    }

    // 阻塞并直到：permitsAvailable
    public void acquire() throws InterruptedException {
```

⊖ ReentrantLock 要求在调用 signal 或 signalAll 时应该持有 Lock，但在 Lock 的具体实现中，在构造 Condition 时也可以不满足这个需求。

```
        lock.lock();
        try {
            while (permits <= 0)
                permitsAvailable.await();
            --permits;
        } finally {
            lock.unlock();
        }
    }

    public void release() {
        lock.lock();
        try {
            ++permits;
            permitsAvailable.signal();
        } finally {
            lock.unlock();
        }
    }
}
```

事实上，它们在实现时都使用了一个共同的基类，即 AbstractQueuedSynchronizer(AQS)，这个类也是其他许多同步类的基类。AQS 是一个用于构建锁和同步器的框架，许多同步器都可以通过 AQS 很容易并且高效地构造出来。不仅 ReentrantLock 和 Semaphore 是基于 AQS 构建的，还包括 CountDownLatch、ReentrantReadWriteLock、SynchronousQueue[⊖]和 FutureTask。

AQS 解决了在实现同步器时涉及的大量细节问题，例如等待线程采用 FIFO 队列操作顺序。在不同的同步器中还可以定义一些灵活的标准来判断某个线程是应该通过还是需要等待。

基于 AQS 来构建同步器能带来许多好处。它不仅能极大地减少实现工作，而且也不必处理在多个位置上发生的竞争问题（这是在没有使用 AQS 来构建同步器时的情况）。在 SemaphoreOnLock 中，获取许可的操作可能在两个时刻阻塞——当锁保护信号量状态时，以及当许可不可用时。在基于 AQS 构建的同步器中，只可能在一个时刻发生阻塞，从而降低上下文切换的开销，并提高吞吐量。在设计 AQS 时充分考虑了可伸缩性，因此 java.util.concurrent 中所有基于 AQS 构建的同步器都能获得这个优势。

14.5 AbstractQueuedSynchronizer

大多数开发者都不会直接使用 AQS，标准同步器类的集合能够满足绝大多数情况的需求。但如果能了解标准同步器类的实现方式，那么对于理解它们的工作原理是非常有帮助的。

在基于 AQS 构建的同步器类中，最基本的操作包括各种形式的获取操作和释放操作。获取操作是一种依赖状态的操作，并且通常会阻塞。当使用锁或信号量时，“获取”操作的含义就很直观，即获取的是锁或者许可，并且调用者可能会一直等待直到同步器类处于可被获取的状态。在使用 CountDownLatch 时，“获取”操作意味着“等待并直到闭锁到达结束状态”，而在使用 FutureTask 时，则意味着“等待并直到任务已经完成”。“释放”并不是一个可阻塞的操

⊖ 在 Java 6 中将基于 AQS 的 SynchronousQueue 替换为一个（可伸缩性更高的）非阻塞的版本。

作，当执行“释放”操作时，所有在请求时被阻塞的线程都会开始执行。

如果一个类想成为状态依赖的类，那么它必须拥有一些状态。AQS 负责管理同步器类中的状态，它管理了一个整数状态信息，可以通过 getState，setState 以及 compareAndSetState 等 protected 类型方法来进行操作。这个整数可以用于表示任意状态。例如，ReentrantLock 用它来表示所有者线程已经重复获取该锁的次数，Semaphore 用它来表示剩余的许可数量，FutureTask 用它来表示任务的状态（尚未开始、正在运行、已完成以及已取消）。在同步器类中还可以自行管理一些额外的状态变量，例如，ReentrantLock 保存了锁的当前所有者的信息，这样就能区分某个获取操作是重入的还是竞争的。

程序清单 14-13 给出了 AQS 中的获取操作与释放操作的形式。根据同步器的不同，获取操作可以是一种独占操作（例如 ReentrantLock），也可以是一个非独占操作（例如 Semaphore 和 CountDownLatch）。一个获取操作包括两部分。首先，同步器判断当前状态是否允许获得操作，如果是，则允许线程执行，否则获取操作将阻塞或失败。这种判断是由同步器的语义决定的。例如，对于锁来说，如果它没有被某个线程持有，那么就能被成功地获取，而对于闭锁来说，如果它处于结束状态，那么也能被成功地获取。

程序清单 14-13 AQS 中获取操作和释放操作的标准形式

```
boolean acquire() throws InterruptedException {
   while (当前状态不允许获取操作) {
          if (需要阻塞获取请求) {
                 如果当前线程不在队列中，则将其插入队列
                 阻塞当前线程
          }
          else
                 返回失败
   }
   可能更新同步器的状态
   如果线程位于队列中，则将其移出队列
   返回成功
}

void release() {
   更新同步器的状态
   if (新的状态允许某个被阻塞的线程获取成功)
          解除队列中一个或多个线程的阻塞状态
}
```

其次，就是更新同步器的状态，获取同步器的某个线程可能会对其他线程能否也获取该同步器造成影响。例如，当获取一个锁后，锁的状态将从“未被持有”变成“已被持有”，而从 Semaphore 中获取一个许可后，将把剩余许可的数量减 1。然而，当一个线程获取闭锁时，并不会影响其他线程能否获取它，因此获取闭锁的操作不会改变闭锁的状态。

如果某个同步器支持独占的获取操作，那么需要实现一些保护方法，包括 tryAcquire、tryRelease 和 isHeldExclusively 等，而对于支持共享获取的同步器，则应该实现 tryAcquireShared 和 tryReleaseShared 等方法。AQS 中的 accuire、acquireShared、release 和 releaseShared

等方法都将调用这些方法在子类中带有前缀try的版本来判断某个操作是否能执行。在同步器的子类中，可以根据其获取操作和释放操作的语义，使用getState、setState以及compareAndSetState来检查和更新状态，并通过返回的状态值来告知基类“获取”或”释放”同步器的操作是否成功。例如，如果tryAcquireShared返回一个负值，那么表示获取操作失败，返回零值表示同步器通过独占方式被获取，返回正值则表示同步器通过非独占方式被获取。对于tryRelease和tryReleaseShared方法来说，如果释放操作使得所有在获取同步器时被阻塞的线程恢复执行，那么这两个方法应该返回true。

为了使支持条件队列的锁（例如ReentrantLock）实现起来更简单，AQS还提供了一些机制来构造与同步器相关联的条件变量。

一个简单的闭锁

程序清单14-14中的OneShotLatch是一个使用AQS实现的二元闭锁。它包含两个公有方法：await和signal，分别对应获取操作和释放操作。起初，闭锁是关闭的，任何调用await的线程都将阻塞并直到闭锁被打开。当通过调用signal打开闭锁时，所有等待中的线程都将被释放，并且随后到达闭锁的线程也被允许执行。

程序清单14-14 使用AbstractQueuedSynchronizer实现的二元闭锁

```
@ThreadSafe
public class OneShotLatch {
    private final Sync sync = new Sync();

    public void signal() { sync.releaseShared(0); }

    public void await() throws InterruptedException {
        sync.acquireSharedInterruptibly(0);
    }

    private class Sync extends AbstractQueuedSynchronizer {
        protected int tryAcquireShared(int ignored) {
            // 如果闭锁是开的（state == 1），那么这个操作将成功，否则将失败
            return (getState() == 1) ? 1 : -1;
        }

        protected boolean tryReleaseShared(int ignored) {
            setState(1);  // 现在打开闭锁
            return true;  // 现在其他的线程可以获取该闭锁

        }
    }
}
```

在OneShotLatch中，AQS状态用来表示闭锁状态——关闭(0)或者打开(1)。await方法调用AQS的acquireSharedInterruptibly，然后接着调用OneShotLatch中的tryAcquireShared方法。在tryAcquireShared的实现中必须返回一个值来表示该获取操作能否执行。如果之前已经打开了闭锁，那么tryAcquireShared将返回成功并允许线程通过，否则就会返回一个表示获取操作失败的值。acquireSharedInterruptibly方法在处理失败的方式，是把这个线程放入

等待线程队列中。类似地，signal 将调用 releaseShared，接下来又会调用 tryReleaseShared。在 tryReleaseShared 中将无条件地把闭锁的状态设置为打开，（通过返回值）表示该同步器处于完全被释放的状态。因而 AQS 让所有等待中的线程都尝试重新请求该同步器，并且由于 tryAcquireShared 将返回成功，因此现在的请求操作将成功。

OneShotLatch 是一个功能全面的、可用的、性能较好的同步器，并且仅使用了大约 20 多行代码就实现了。当然，它缺少了一些有用的特性，例如限时的请求操作以及检查闭锁的状态，但这些功能实现起来同样很容易，因为 AQS 提供了限时版本的获取方法，以及一些在常见检查中使用的辅助方法。

oneShotLatch 也可以通过扩展 AQS 来实现，而不是将一些功能委托给 AQS，但这种做法并不合理 [EJ Item 14]，原因有很多。这样做将破坏 OneShotLatch 接口（只有两个方法）的简洁性，并且虽然 AQS 的公共方法不允许调用者破坏闭锁的状态，但调用者仍可以很容易地误用它们。java.util.concurrent 中的所有同步器类都没有直接扩展 AQS，而是都将它们的相应功能委托给私有的 AQS 子类来实现。

14.6 java.util.concurrent 同步器类中的 AQS

java.util.concurrent 中的许多可阻塞类，例如 ReentrantLock、Semaphore、ReentrantReadWriteLock、CountDownLatch、SynchronousQueue 和 FutureTask 等，都是基于 AQS 构建的。我们快速地浏览一下每个类是如何使用 AQS 的，不需要过于地深入了解细节（在 JDK 的下载包中包含了源代码[⊖]）。

14.6.1 ReentrantLock

ReentrantLock 只支持独占方式的获取操作，因此它实现了 tryAcquire、tryRelease 和 isHeldExclusively，程序清单 14-15 给出了非公平版本的 tryAcquire。ReentrantLock 将同步状态用于保存锁获取操作的次数，并且还维护一个 owner 变量来保存当前所有者线程的标识符，只有在当前线程刚刚获取到锁，或者正要释放锁的时候，才会修改这个变量[⊜]。在 tryRelease 中检查 owner 域，从而确保当前线程在执行 unlock 操作之前已经获取了锁：在 tryAcquire 中将使用这个域来区分获取操作是重入的还是竞争的。

程序清单 14-15 基于非公平的 ReentrantLock 实现 tryAcquire

```
protected boolean tryAcquire(int ignored) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        if (compareAndSetState(0, 1)) {
```

⊖ 也可以从 http://gee.cs.oswego.edu/dl/concurrency-interest 获得，只是存在一些许可限制。

⊜ 由于受保护的状态操作方法具有 volatile 类型的内存读写语义，同时 ReentrantLock 只是在调用 getState 之后才会读取 owner 域，并且只有在调用 setState 之前才会写入 owner，因此 ReentrantLock 可以拥有同步状态的内存语义，因此避免了进一步的同步（请参见 16.1.4 节）。

```
                owner = current;
                return true;
            }
        } else if (current == owner) {
            setState(c+1);
            return true;
        }
        return false;
    }
```

当一个线程尝试获取锁时，tryAcquire 将首先检查锁的状态。如果锁未被持有，那么它将尝试更新锁的状态以表示锁已经被持有。由于状态可能在检查后被立即修改，因此 tryAcquire 使用 compareAndSetState 来原子地更新状态，表示这个锁已经被占有，并确保状态在最后一次检查以后就没有被修改过。（请参见 15.3 节中对 compareAndSet 的描述）。如果锁状态表明它已经被持有，并且如果当前线程是锁的拥有者，那么获取计数会递增，如果当前线程不是锁的拥有者，那么获取操作将失败。

ReentrantLock 还利用了 AQS 对多个条件变量和多个等待线程集的内置支持。Lock.newCondition 将返回一个新的 ConditionObject 实例，这是 AQS 的一个内部类。

14.6.2 Semaphore 与 CountDownLatch

Semaphore 将 AQS 的同步状态用于保存当前可用许可的数量。tryAcquireShared 方法（请参见程序清单 14-16）首先计算剩余许可的数量，如果没有足够的许可，那么会返回一个值表示获取操作失败。如果还有剩余的许可，那么 tryAcquireShared 会通过 compareAndSetState 以原子方式来降低许可的计数。如果这个操作成功（这意味着许可的计数自从上一次读取后就没有被修改过），那么将返回一个值表示获取操作成功。在返回值中还包含了表示其他共享获取操作能否成功的信息，如果成功，那么其他等待的线程同样会解除阻塞。

程序清单 14-16 Semaphore 中的 tryAcquireShared 与 tryReleaseShared

```
protected int tryAcquireShared(int acquires) {
    while (true) {
        int available = getState();
        int remaining = available - acquires;
        if (remaining < 0
                || compareAndSetState(available, remaining))
            return remaining;
    }
}

protected boolean tryReleaseShared(int releases) {
    while (true) {
        int p = getState();
        if (compareAndSetState(p, p + releases))
            return true;
    }
}
```

当没有足够的许可，或者当 tryAcquireShared 可以通过原子方式来更新许可的计数以响应获取操作时，while 循环将终止。虽然对 compareAndSetState 的调用可能由于与另一个线程发生竞争而失败（请参见 15.3 节），并使其重新尝试，但在经过了一定次数的重试操作以后，在这两个结束条件中有一个会变为真。同样，tryReleaseShared 将增加许可计数，这可能会解除等待中线程的阻塞状态，并且不断地重试直到更新操作成功。tryReleaseShared 的返回值表示在这次释放操作中解除了其他线程的阻塞。

CountDownLatch 使用 AQS 的方式与 Semaphore 很相似：在同步状态中保存的是当前的计数值。countDown 方法调用 release，从而导致计数值递减，并且当计数值为零时，解除所有等待线程的阻塞。await 调用 acquire，当计数器为零时，acquire 将立即返回，否则将阻塞。

14.6.3 FutureTask

初看上去，FutureTask 甚至不像一个同步器，但 Future.get 的语义非常类似于闭锁的语义——如果发生了某个事件（由 FutureTask 表示的任务执行完成或被取消），那么线程就可以恢复执行，否则这些线程将停留在队列中并直到该事件发生。

在 FutureTask 中，AQS 同步状态被用来保存任务的状态，例如，正在运行、已完成或已取消。FutureTask 还维护一些额外的状态变量，用来保存计算结果或者抛出的异常。此外，它还维护了一个引用，指向正在执行计算任务的线程（如果它当前处于运行状态），因而如果任务取消，该线程就会中断。

14.6.4 ReentrantReadWriteLock

ReadWriteLock 接口表示存在两个锁：一个读取锁和一个写入锁，但在基于 AQS 实现的 ReentrantReadWriteLock 中，单个 AQS 子类将同时管理读取加锁和写入加锁。ReentrantReadWriteLock 使用了一个 16 位的状态来表示写入锁的计数，并且使用了另一个 16 位的状态来表示读取锁的计数。在读取锁上的操作将使用共享的获取方法与释放方法，在写入锁上的操作将使用独占的获取方法与释放方法。

AQS 在内部维护一个等待线程队列，其中记录了某个线程请求的是独占访问还是共享访问。在 ReentrantReadWriteLock 中，当锁可用时，如果位于队列头部的线程执行写入操作，那么线程会得到这个锁，如果位于队列头部的线程执行读取访问，那么队列中在第一个写入线程之前的所有线程都将获得这个锁。[⊖]

⊖ 这种机制并不允许选择读取线程优先或写入线程优先等策略，在某些读写锁实现中也采用了这种方式。因此，要么 AQS 的等待队列不能是一个 FIFO 队列，要么使用两个队列。然而，在实际中很少需要这么严格的排序策略。如果非公平版本的 ReentrantReadWriteLock 无法提供足够的活跃性，那么公平版本的 ReentrantReadWriteLock 通常会提供令人满意的排序保证，并且能确保读取线程和写入线程不会发生饥饿问题。

小结

要实现一个依赖状态的类——如果没有满足依赖状态的前提条件，那么这个类的方法必须阻塞，那么最好的方式是基于现有的库类来构建，例如 Semaphore.BlockingQueue 或 CountDownLatch，如第 8 章的 ValueLatch 所示。然而，有时候现有的库类不能提供足够的功能，在这种情况下，可以使用内置的条件队列、显式的 Condition 对象或者 AbstractQueuedSynchronizer 来构建自己的同步器。内置条件队列与内置锁是紧密绑定在一起的，这是因为管理状态依赖性的机制必须与确保状态一致性的机制关联起来。同样，显式的 Condition 与显式的 Lock 也是紧密地绑定到一起的，并且与内置条件队列相比，还提供了一个扩展的功能集，包括每个锁对应于多个等待线程集，可中断或不可中断的条件等待，公平或非公平的队列操作，以及基于时限的等待。